

Day 06

OpenSTA and clock constraints

VLSI Design Flow RTL to GDS

Lessons 31 to 32

Timing reports and clock constraints through Week 8 Constraints I.

Contents and bookmarks link to sections; figures and page footers return to contents.

Course: Prof. Sneha Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Worked examples state their analytical assumptions.

Contents

Lesson 31 Static timing analysis using OpenSTA	2
Read a timing report as a calculation	2
A report-shaped example	3
Lesson 32 Constraints I	4
B-09: Constraint categories and primary-clock definitions	5
B-10: Generated clocks, latency and model versus hardware	9
B-11: Jitter, skew, uncertainty and transition	11
Scope of Constraints I	13
Sources and references	14
Primary references for the technical clarifications	14

Lesson 31 Static timing analysis using OpenSTA

Week 7 · [Lecture video](#)

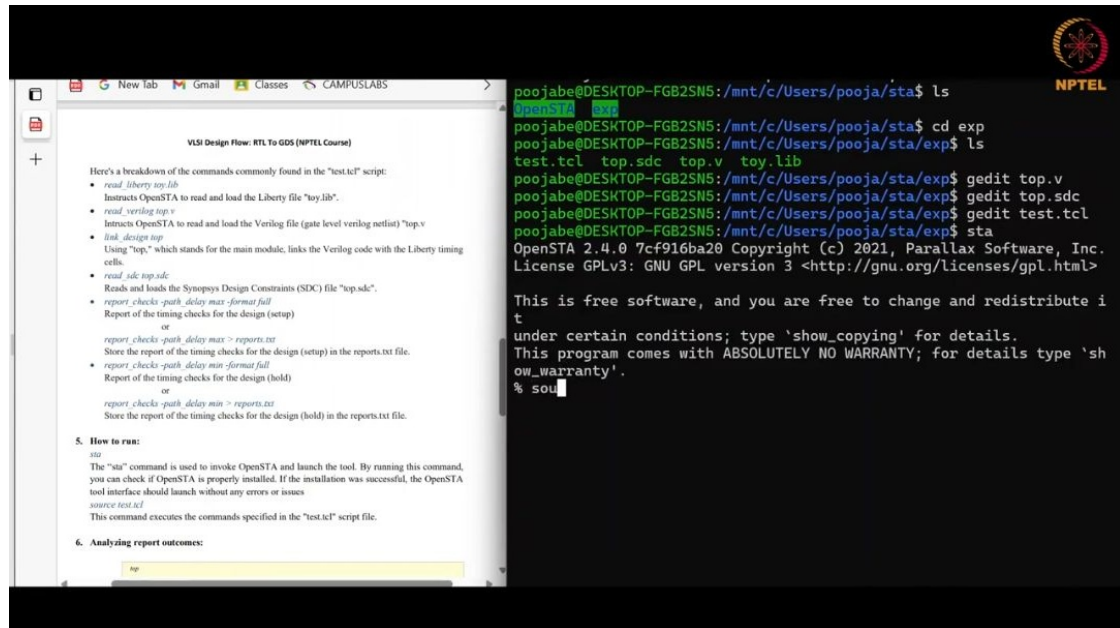


Figure 6.01. Lecture: [Lecture at 8:20](#).

Read a timing report as a calculation

OpenSTA is a static timing analyzer. The tutorial loads timing models and a netlist, establishes design and constraints, and reports paths. It analyzes permitted transitions using timing models rather than running one input-vector waveform. Its result is conditional on the supplied clock, environment, cell and wire assumptions.

```
Tcl
read_liberty toy.lib
read_verilog mapped.v
link_design top
read_sdc top.sdc
report_units
check_setup
report_checks -path_delay max -format full_clock_expanded
report_checks -path_delay min -format full_clock_expanded
```

This script assumes a matching mapped netlist, Liberty file and SDC file in its working directory. `link_design` resolves instantiated cell references. `report_units` identifies the numeric scale; `check_setup` diagnoses missing analysis requirements. The max-path report is read for setup, and the min-path report for hold. Consult the [OpenSTA commands](#) for options supported by the installed version.

For each reported path, identify the startpoint, endpoint, launch/capture clocks and edges, clock-path contributions, clock-to-Q, every cell/net increment, data arrival, data required time, uncertainty and slack. Recompute the final subtraction. An increment is the contribution of one step; the path column is the accumulated arrival. A hold report's slack uses the earliest arrival minus the hold requirement, so its arithmetic differs from setup.

A report-shaped example

Assume all numbers below are ns, with $T=10$, launch latency 1, capture latency 1.5, $tCQ_{max}=0.8$, data delay=5.2, setup=0.7 and setup uncertainty=0.3. Latest arrival is $1+0.8+5.2=7.0$; required arrival is $10+1.5-0.7-0.3=10.5$; setup slack is 3.5. For $tCQ_{min}=0.2$, $d_{min}=0.4$, hold=0.25 and hold uncertainty=0.1, earliest arrival is 1.6, required hold arrival is $1.5+0.25+0.1=1.85$, and hold slack is -0.25.

The same path therefore passes this setup check and fails this hold check. Reading only the worst max-path report would miss the violation. A lower clock frequency would increase setup margin but would leave this same-edge hold comparison unchanged.

Setup-report contribution	Increment or operation (ns)	Accumulated time (ns)
Launch clock at the register	+1.0	1.0
Clock-to-Q	+0.8	1.8
Data cells and nets	+5.2	7.0 arrival
Next capture edge plus capture latency	$10.0 + 1.5$	11.5
Setup requirement and uncertainty	$-0.7 - 0.3$	10.5 required
Setup slack	$10.5 - 7.0$	+3.5

The required-time calculation starts a separate clock-side accumulation; it is not another data-path increment added to 7.0. A report's labels determine which column and check each contribution belongs to. The hold calculation separately uses minimum clock-to-Q and data increments and forms arrival minus required time.

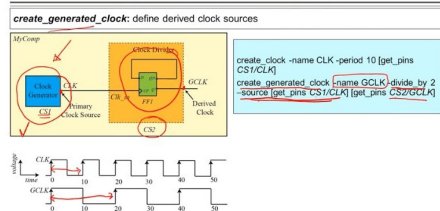
The absence of a path from a report can mean it is unconstrained, disconnected, excluded by a justified exception, or outside the selected reporting filter. It does not automatically mean it has no timing problem. Missing parasitics also change what the result represents: an early estimated-wire analysis should not be presented as extracted post-route signoff.

Lesson 32 Constraints I

Week 8 · [Lecture video](#)

Lecture

Derived Clock Source Definition

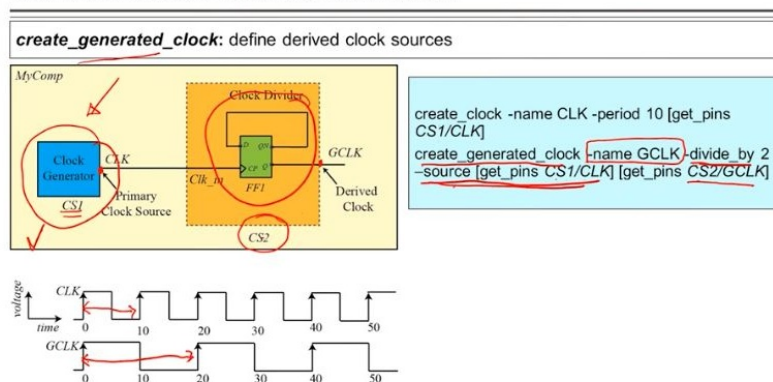


Handwritten notes

`create_clock -name INT.CLK -period 10 [get_pins CS1/CLK-g]` ^{CS1}
`create_generated_clock`
`create_clock -name CLK -period 10 [get_pins CS1/CLK]` Tool will
`create_generated_clock -name GCLK` understand that
 CLK-g is a special pin, generating
 a clk with a clock time period of
 10.
 - divide by 2
 - source [get_pins CS1/CLK]
 [get_pins CS2/GCLK]
 Time unit will be there in
 library -
 set unit - for time unit -
 create_clock -
 not creating/implementing

Lecture: [38:20](#) · Source: B-10.

Derived Clock Source Definition



10



Figure 6.04. Lecture: [38:20](#).

B-09: Constraint categories and primary-clock definitions

Constraint is written by designer -
 - Tools for automatic constraint generation.

Constraints	clock	Environment	functionality
Types	- duty - skew - delay	- meaning - signal - behaviour etc.	

- Design Rules
 - Max Capacitance
 - Max skew

① Clock constraints :: Clock sources.

- Primary sources - waveform independent of other clock sources.
- Waveform depends on other sources.

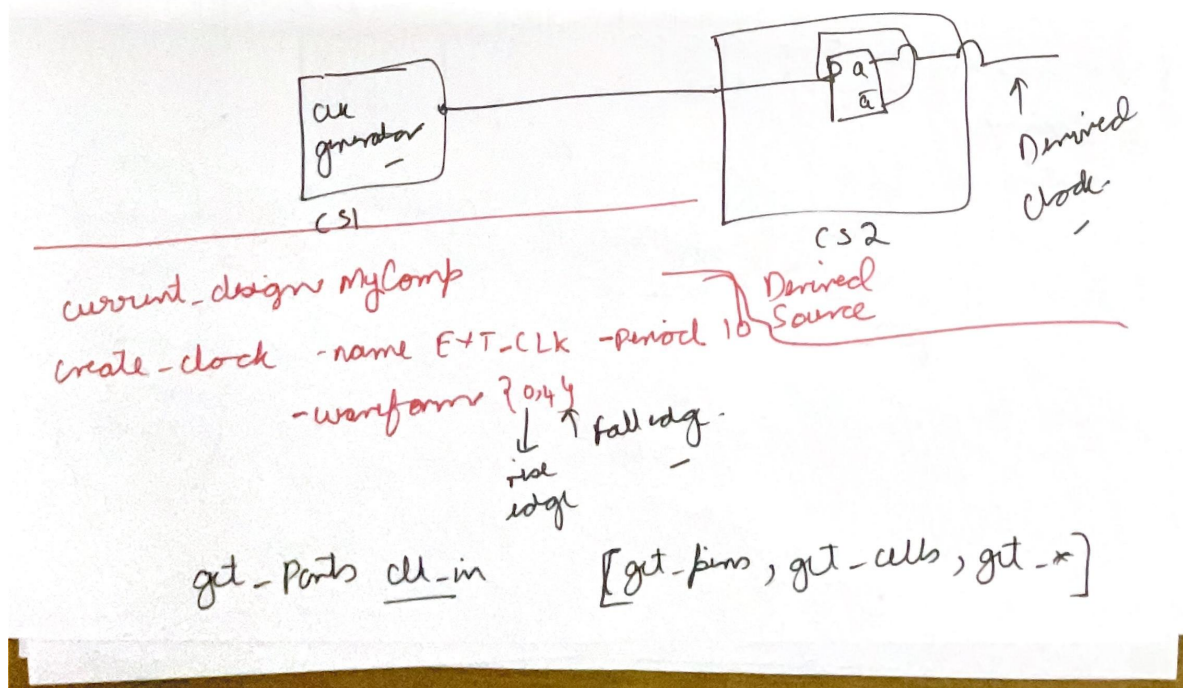


Figure 6.05. Source: Scan B, PDF page 9.

Requirements, analysis and the clock model

Concept	Technical interpretation
Constraints describe the requirements under which synthesis and STA operate.	B-09 distinguishes clocks, environment, functionality and design rules. A standalone timing analyzer reports behavior; synthesis or physical optimization may modify the circuit.
Primary and generated clocks preserve different timing relationships.	B-10 records the source and target of a generated clock. A clock command defines a model; it does not build a clock generator.
Latency, transition and uncertainty have separate roles.	B-10 and B-11 distinguish clock arrival, waveform shape and margin. Setup and hold consume uncertainty on opposite sides of their inequalities.

Constraints convey design requirements and environmental assumptions to tools. They may be written manually or generated from higher-level requirements, but consistency still needs review. A constraint that refers to a nonexistent port or the wrong clock describes the wrong analysis, even if the file parses successfully.

The four categories have distinct meanings. Clock constraints describe sources, periods, waveforms and clock properties. Environment constraints describe influences such as input drive or transition and output loading. Functionality constraints describe operational assumptions or mode choices. Design rules constrain properties such as maximum transition and capacitance. A timing slack can pass while a transition or capacitance rule fails; the rule checks are separate.

The clock-generator diagram distinguishes a primary clock from a derived clock. A primary clock waveform is specified independently. A generated clock derives its waveform relationship from an existing master clock. It can be divided, multiplied, inverted or otherwise related through supported definitions. Merely declaring two independent primary clocks with matching periods does not express a generated relationship.

```
Tcl
current_design MyComp
create_clock -name EXT_CLK -period 10 \
  -waveform {0 4} [get_ports clk_in]
```

The waveform has a rising edge at zero and falling edge at four in every ten-unit period, so its duty cycle is 40%. It is not a 4-unit period. Square brackets perform Tcl command substitution: `get_ports` returns the design objects to which the outer command applies. `get_pins`, `get_cells`, `get_nets` and `get_clocks` select different object kinds. Their spellings use underscores, correcting the hyphenated handwritten command forms.

The numbers are in the tool's active time units. Read the library and command-unit setup rather than assuming “10” always means 10 ns. Hierarchical pin names such as `CS1/clk_g` must match the linked design. The [OpenSTA command reference](#) documents clock creation and object selection.

Waveform edges determine the available interval

For the specified waveform, rising edges occur at 0, 10 and 20 time units; falling edges occur at 4, 14 and 24. A rising-edge launch at zero followed by a falling-edge capture at four has four units of nominal setup separation. A falling-edge launch at four followed by a rising-edge capture at ten has six. Neither mixed-edge path automatically receives the full ten-unit period, and a 40% duty cycle does not produce two equal half-period budgets.

For the first pair, assume launch latency 0.2, capture latency 0.4, maximum clock-to-Q 0.3, data delay 2.5, setup 0.5 and setup uncertainty 0.2 in the same time unit. Arrival is $0.2 + 0.3 + 2.5 = 3.0$; required time is $4 + 0.4 - 0.5 - 0.2 = 3.7$; slack is $+0.7$. Substituting the period 10 for the actual capture edge would overstate the slack by six units. Edge selection comes before the usual arrival/deadline subtraction.

A period alone does not specify pulse widths

The same waveform has a four-unit high interval and six-unit low interval. Suppose a destination cell requires a minimum high pulse of 4.5 and a minimum low pulse of 3.0 in these units. The modeled high-pulse margin is $4 - 4.5 = -0.5$, while the low-pulse margin is $6 - 3 = +3$. Passing setup and hold alone does not establish that this separate clock requirement passes.

A waveform with edges at 0 and 5 would have the same period but different high/low durations and mixed-edge setup budgets. Such a model is valid only if it represents the intended clock. Altering a waveform declaration to improve a report does not change the physical pulse delivered to a register. The clock definition in B-09 supplies event locations; the characterized destination requirements determine whether those events and pulse durations are acceptable.

Derive the hold pairing for the same mixed-edge paths

For the existing ten-unit clock with rising edges at 0 and 10 and falling edges at 4 and 14, a rising-to-falling path launches old data at 0 for capture at 4. Hold asks whether the next data launched at 10 can reach that destination too soon after its capture at 4. The source and destination edges are separated by six units for this hold comparison. Equivalently, translate both edges back one period and compare launch at 0 with the preceding falling capture at -6.

For a falling-to-rising path, old data launches at 4 for capture at 10. The next falling launch at 14 is four units after that capture, giving the complementary hold separation. The same edge schedule must be used consistently; substituting the setup separation into the hold comparison swaps these two gaps.

To include latency and uncertainty, let $E_{L,next}$ be the nominal next-data launch edge and E_C the nominal capture edge. Let L and C be their respective clock latencies and u_H the positive hold uncertainty. Then:

$$A_{new,min} = E_{L,next} + L + t_{CQ,min} + d_{min}$$

$$R_{hold} = E_C + C + t_H + u_H$$

Hold slack is $A_{new,min} - R_{hold}$. The new data must arrive at or after the protected boundary. Positive $C - L$ moves capture later relative to launch and reduces this margin; increasing u_H extends the protected interval and also reduces it. Both signs follow from the absolute event times.

For a numerical comparison, assume ideal zero-latency clocks, no uncertainty, maximum clock-to-Q 0.20, maximum data delay 2.90, setup 0.30, minimum clock-to-Q 0.08, minimum data delay 0.12 and

hold 0.10, all in the same arbitrary time unit. These assumptions differ from the preceding skewed-clock example and isolate edge geometry.

Path	Setup separation	Setup slack	Next-launch gap after capture	Hold slack
Rising to falling	4	$4 - 0.20 - 2.90 - 0.30 = +0.60$	6	$6 + 0.08 + 0.12 - 0.10 = +6.10$
Falling to rising	6	$6 - 0.20 - 2.90 - 0.30 = +2.60$	4	$4 + 0.08 + 0.12 - 0.10 = +4.10$

The hold margins are large because the next launching edge is several units later than capture, not because the combinational path is intrinsically slow.

Draw the old-data launch, capture and next-data launch before substituting numbers. Setup uses the latest old-data arrival before capture; hold uses the earliest new-data arrival after capture. A report's edge pairing and data-event identity are part of the calculation, rather than labels that can be omitted once the delays are known.

B-10: Generated clocks, latency and model versus hardware

`create-clock -name INT-CLK -period 10 [get-pins CS1/clk-g]` ^(S4)
`create-generated-clock`
`create-clock -name CLK -period 10 [get-pins CS1/CLK]` Tool will
`create-generated-clock -name GCLK` understand that
 - divide-by 2
 - source [get-pins CS1/CLK]
 [get-pins CS2/GCLK]

CLK-g is a special pin, generating a clk with a clock time period of 10.

Time unit will be there in library -
 set-unit - for time unit -
 create-clock -
 Tool is not creating/implementing a clock generator in the design.

0 time -
 Source latency = 5 unit
 n/w latency = 10 (in n/w in our design).

`create-clock -name CLK -period 200 [get-clock ports CLK-Port]`
`set-clock-latency 5 -source [get-clock CLK]`
 Attributes of clock signal: uncertainty
 set-clock-uncertainty
 unpredictable deviation of the clock edges from the ideal value.

Figure 6.06. Source: Scan B, PDF page 10.

The top command declares an internal primary clock at an output pin of CS1; the following command describes a clock derived from a master. In the course example:

```
Tcl
create_clock -name CLK -period 10 [get_pins CS1/CLK]
create_generated_clock -name GCLK -divide_by 2 \
    -source [get_pins CS1/CLK] [get_pins CS2/GCLK]
```

GCLK has a period of 20 active time units, maintaining its relationship to the CLK source. `-source` names the master waveform reference point; the final object collection names the generated clock's target. The command does not instantiate a divider. A real divider must already be described or implemented, and the timing model must match its actual behavior.

A generated period is not every path's setup budget

Consider a phase-aligned divide-by-two implementation with master rising edges at 0, 10, 20 and 30, and generated rising edges at 0, 20 and 40. This is an explicit waveform assumption for the example. For a master-clocked source register feeding a generated-clocked destination, the master can launch new data at 10 before capture at 20. That edge pair has ten units of nominal separation, even though GCLK's period is twenty.

With clock-to-Q plus data delay 7 and setup requirement 1, under zero latency and uncertainty, the latest data from launch 10 arrives at 17 and must arrive by 19, leaving slack +2. Granting a twenty-unit budget would incorrectly add ten units of margin. A new master launch at 20 also coincides with the generated capture edge, requiring the corresponding earliest-data hold check. The generated-clock relationship supplies these related edges; the timing model and chosen divider phase must agree with the implemented clock behavior. The generated-clock reference specifies source and waveform relationships.

The lower drawing separates source latency from network latency. Source latency is the delay from the original clock source to the defined clock point; network latency is from that definition point through the clock distribution to a sequential clock pin. In the illustrated estimates, source is 5 and network is 10, giving 15 units from original source to the clock pin.

```
Tcl
create_clock -name clk -period 200 [get_ports clk_port]
set_clock_latency -source 5 [get_clocks clk]
set_clock_latency 10 [get_clocks clk]
```

These are ideal-clock latency estimates. Once a real clock network is propagated, its modeled delay replaces the corresponding network estimate under the tool's clock-analysis rules. Avoid adding an estimate again to an already propagated path. Common source latency may cancel in some same-clock register comparisons, but absolute source timing and inter-clock relationships still require correct modeling.

For a same-clock path with common source latency S and network contributions N_L and N_C , launch and capture latencies are $L = S + N_L$ and $C = S + N_C$. Their skew is $C - L = N_C - N_L$. With $S = 5$, $N_L = 10$ and $N_C = 12$, the absolute arrivals are 15 and 17 but the relative skew is two units. Raising S to eight changes those arrivals to 18 and 20 while leaving skew at two. This cancellation assumes a genuinely common source contribution; unrelated clocks, different modeled edge effects or distinct source paths cannot be collapsed by the same argument.

The handwritten “set_unit” is not a universal SDC command. Verify the selected analyzer's supported unit commands; for OpenSTA, inspect report_units and its documented command-unit controls. The important principle is that the library/header and active tool units give numeric meaning, rather than a guessed suffix absent from the SDC file.

B-11: Jitter, skew, uncertainty and transition

Attributes of Clock Signal: Uncertainty

set_clock_uncertainty: unpredictable deviation of the clock edges from the ideal value

Clock uncertainty can be used to model:

- Jitter: temporal variation
- Skew: spatial variation
- Safety margins

Figure 6.07. Lecture: [Clock uncertainty at 49:20](#). Jitter, skew estimates and explicit margins have different physical origins.

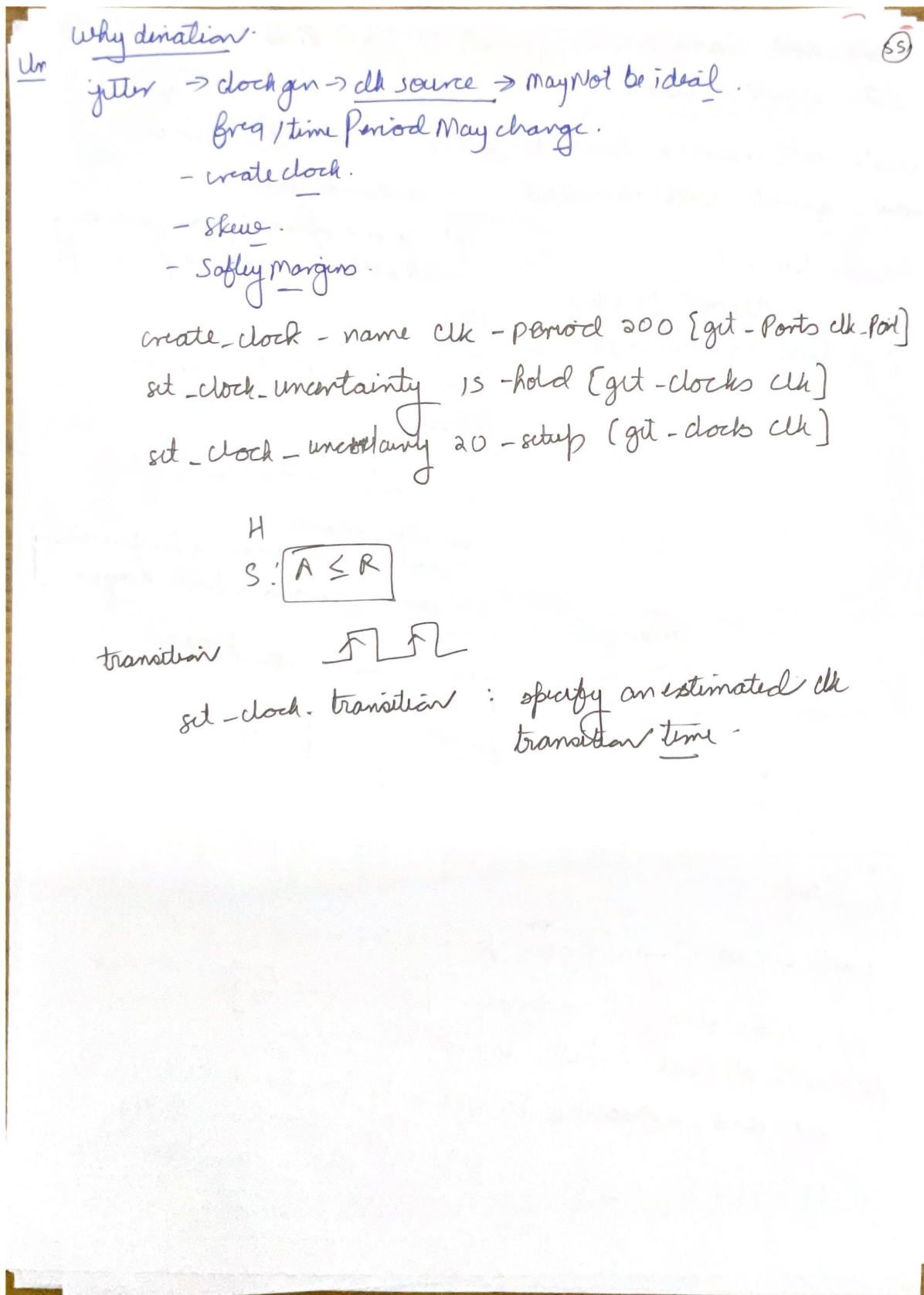


Figure 6.08. Source: Scan B, PDF page 11.

Jitter describes variation of clock edge timing relative to its intended timing; skew is the difference in arrival of a related clock edge at different points. They are separate phenomena. Clock uncertainty is a timing margin that may model jitter and explicitly reserved effects. It does not shift the nominal waveform in the same way as latency.

The course uses a 200-unit period with different uncertainty for setup and hold:

```
Tcl
create_clock -name clk -period 200 [get_ports clk_port]
set_clock_uncertainty -hold 15 [get_clocks clk]
set_clock_uncertainty -setup 20 [get_clocks clk]
```

For the same-edge/next-edge relationships used in these notes, setup requires $L+tCQ_{max}+d_{max} \leq T+C-tSU-U_{setup}$. Hold requires $L+tCQ_{min}+d_{min} \geq C+tH+U_{hold}$. A positive uncertainty tightens both checks: it reduces the setup deadline and raises the earliest permitted hold arrival. The isolated $A \leq R$ box in the handwriting applies to setup; hold uses $A \geq R$.

For zero latency and $tCQ_{max}=10$, $d_{max}=150$, $tSU=10$, setup uncertainty 20, the setup arrival is 160 and deadline 170, giving slack +10. Without the uncertainty, the slack would be +30. For $tCQ_{min}=5$, $d_{min}=12$, $tH=3$ and hold uncertainty 15, earliest arrival is 17 and the hold threshold is 18, giving slack -1. Both examples use the same arbitrary time unit as the 200-unit period.

The transition annotation specifies an estimated clock rise/fall duration, not a period or edge-location margin. A separate lecture example uses a 2000-unit period and transition 10. Keep that scale separate from the 200-unit uncertainty example. For an ideal clock model, the transition helps select library requirements; a propagated clock obtains transitions from its modeled distribution.

Recall check: changing `set_clock_transition` does not add a buffer to the netlist, and changing `set_clock_uncertainty` does not create physical jitter. They change the analysis model. Explain the actual source of any margin before treating its removal as a design improvement.

Scope of Constraints I

The present scope covers OpenSTA and Constraints I, through course Lesson 32.

The Constraints I lecture-material deck covers the constraint categories and clock definitions/properties. Its agenda previews other timing requirements, but detailed I/O-delay and timing-exception notes belong to the next lecture.

Sources and references

Primary course: **VLSI Design Flow: RTL to GDS**, Prof. Sneha Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Lecture captions provide video timestamps. Additional derivations and practice examples are study annotations.

[Open the course video playlist.](#)

Original handwritten sources: **Scan B PDF pages 9 to 11**. Source identifiers are separate from the page numbers of this document.

Primary references for the technical clarifications

1. [OpenSTA commands](#)
2. [OpenSTA commands](#)
3. [OpenSTA commands](#)

Numerical examples use stated teaching assumptions. Technology-specific library values and manufacturing rules require their corresponding source models. Verilog examples use traditional Verilog semantics unless otherwise stated.